

`std::priority_queue`

From STL Basics to Systems-Level Reasoning

C++ Data Structures for High-Performance Systems

"What should happen next?"

You need to repeatedly retrieve the best next item from a changing set.

Sorted vector?

$O(n)$ insert every time.
Re-sorting after each change
is wasteful when you only
need the best.

`std::set` / tree?

$O(\log n)$ insert + extract,
but scattered memory, pointer
chasing, high constant factors.
Pays for features you may not need.

Binary heap?

$O(\log n)$ insert + extract,
contiguous memory, cache-friendly,
tiny constant factor.
Optimized for exactly this problem.

A priority queue is the data structure built for this question.



What Is `std::priority_queue`?

- A container adaptor — not a standalone container
- Wraps an underlying container (default: `std::vector`)
- Maintains a max-heap invariant at all times
- Provides: `push()`, `emplace()`, `pop()`, `top()`, `empty()`, `size()`
- `top()` always returns the highest-priority element
- No iterators, no random access, no ordered traversal

```
#include <queue>

// Template signature:
// priority_queue<T, Container, Compare>

std::priority_queue<int> pq; // max-heap

pq.push(10);
pq.push(30);
pq.push(20);
pq.emplace(25); // construct in-place, no copy

pq.top(); // 30 — the maximum
pq.pop(); // removes 30
pq.top(); // 20
```

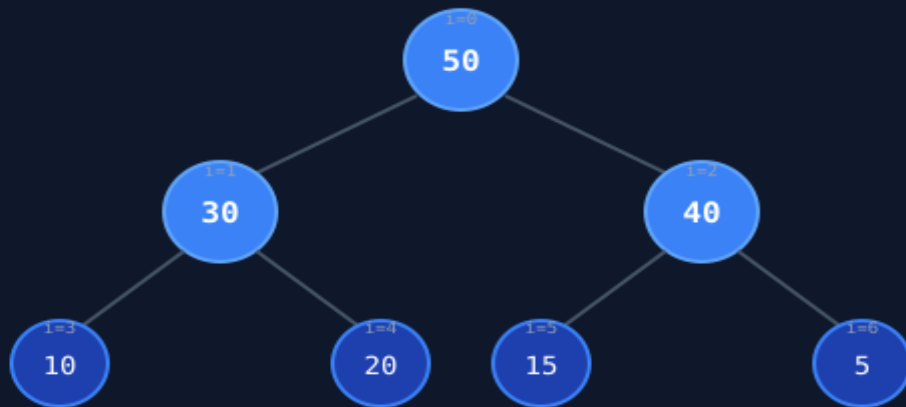


Container Adaptor: `priority_queue` doesn't own storage — it delegates to `vector` (or `deque`). It adds heap logic on top. The restricted API is a feature: it prevents you from violating the heap invariant. Use `emplace()` (C++11) over `push()` for heavy objects — it constructs in-place, avoiding copies. `push()` also supports move semantics via `push(std::move(x))`.



The Binary Heap: Array-Backed Tree

Binary Max-Heap (Tree View)



Array-Backed Storage (contiguous memory)



Index Formulas

$\text{parent}(i) = (i-1)/2$

$\text{left}(i) = 2*i + 1$

$\text{right}(i) = 2*i + 2$

Cache-friendly:

Elements stored in a single contiguous memory block

`push sift UP  $O(\log n)$`

`pop sift DOWN  $O(\log n)$`

`top arr[0]  $O(1)$`

Key insight: A binary heap is a complete binary tree stored in a flat array. No pointers, no per-node allocation. Parent/child relationships use index arithmetic, not pointer chasing.

Heap Order vs Sorted Order

Fully Sorted Array



Every element $\geq$ the one to its right. Full global ordering.

Heap-Ordered Array (NOT sorted!)



Guaranteed max

30 appears before 40 – only parent $\geq$ children invariant holds, not full sort

Iterating the underlying array does NOT give you sorted output!



top() guarantees one thing: the element at position [0] is the maximum under the comparator. It guarantees nothing about the order of any other element. Do not iterate the underlying container expecting sorted output — that is the most common conceptual mistake with heaps.

⇌ Push & Pop: Sift-Up and Sift-Down

push(45) : Sift Up

1. Insert at end

50	30	40	10	20	15	45
----	----	----	----	----	----	----
2. $45 > \text{parent}(40)$? Swap!

50	30	45	10	20	15	40
----	----	----	----	----	----	----
3. $45 > \text{parent}(50)$? No. Done

50	30	45	10	20	15	40
----	----	----	----	----	----	----



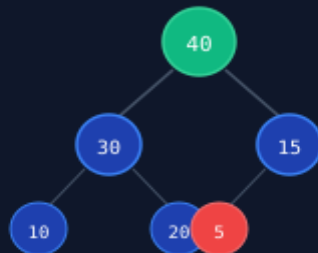
pop() : Sift Down

1. Move last to root

5	30	40	10	20	15
---	----	----	----	----	----
2. $5 < \max \text{child}(40)$? Swap!

40	30	5	10	20	15
----	----	---	----	----	----
3. $5 < \max \text{child}(15)$? Swap!

40	30	15	10	20	5
----	----	----	----	----	---



Why $O(\log n)$?

A complete binary tree with n nodes has height $\text{floor}(\log_2(n))$

Sift-up and sift-down traverse at most one root-to-leaf path

$n = 1,000,000$ elements = at most 20 comparisons per push/pop

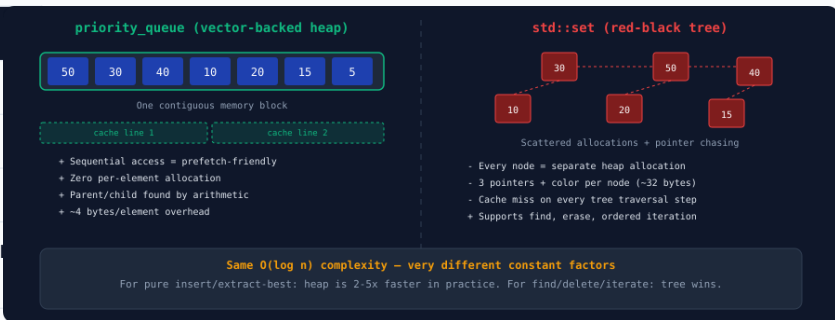
`top()` just reads `arr[0]`: always $O(1)$, zero traversal

Performance: For $n = 1,000,000$ elements: at most 20 comparisons per push or pop. `top()` just reads `arr[0]` — always $O(1)$, zero traversal.



Complexity & Performance Beyond Big-O

Operation	Complexity	Notes
push()	$O(\log n)$	Amortized — vector may resize
pop()	$O(\log n)$	Sift-down from root to leaf
top()	$O(1)$	Direct array access — arr[0]
make_heap	$O(n)$	Bottom-up construction — cheaper than n pushes
size() /	$O(1)$	Delegated to underlying container



Why Heaps Win on Constant Factors

Contiguous memory

All elements in one block — CPU prefetcher loves this

No pointer chasing

Parent/child found by arithmetic, not dereference

No per-node allocation

push() calls push_back — amortized $O(1)$ due to vector doubling; reserve(n) if size known

Small constant factor

Each level: one compare + one swap — nothing else

Branch prediction

Sift-down tends to go all the way down — predictable path



Comparator Semantics: The Mental Model

Default: Max-Heap

```
std::priority_queue<int>  
// uses std::less<int>  
// less(a,b) → a has LOWER  
priority  
// → largest rises to top()
```

Min-Heap

```
std::priority_queue<int,  
    vector<int>,  
    greater<int>>  
// greater(a,b) → a has LOWER  
priority  
// → smallest rises to top()
```

Custom

```
auto cmp = [](const Job& a,  
              const Job& b){  
    return a.cost > b.cost;  
}; // min by cost  
  
priority_queue<Job,  
    vector<Job>,  
    decltype(cmp)> pq(cmp);
```



One rule to remember: The comparator answers "does a have LOWER priority than b?" If `cmp(a, b)` returns true, a sinks and b rises.

So `std::less` (default) means smaller values have lower priority → largest on top (max-heap).
`std::greater` means larger values have lower priority → smallest on top (min-heap).
The element at `top()` is the one considered "largest" under the queue's ordering rule.



Comparator Correctness & Tie-Breaking

Wrong Intuition

```
// BUG: uses <= (violates irreflexivity → UB)
auto bad = [](int a, int b){ return a <= b; };

// BUG: no tiebreaker → non-deterministic order
auto bad2 = [](const Order& a, const Order& b){
    return a.price < b.price; // ties are random
};
```

Strict Weak Ordering Rules

- Irreflexivity: $\text{comp}(a, a)$ must be false
- Asymmetry: $\text{comp}(a, b) \rightarrow \neg \text{comp}(b, a)$
- Transitivity: $\text{comp}(a, b) \wedge \text{comp}(b, c) \rightarrow \text{comp}(a, c)$

Correct Version

```
// GOOD: strict < only
auto good = [](int a, int b){ return a < b; };

// GOOD: deterministic tiebreaker
auto good2 = [](const Order& a, const Order& b){
    if (a.price != b.price) return a.price < b.price;
    return a.seq_num > b.seq_num; // FIFO on tie
};
```

```
// Production-grade comparison
struct LimitOrder { // price, arrival_seq, order_id
    bool operator<(const LimitOrder& o) const {
        if (price != o.price) return price < o.price;
        if (arrival_seq != o.arrival_seq)
            return arrival_seq > o.arrival_seq; //FIFO
        return order_id > o.order_id;
    }
};
```

Why tiebreakers matter: Without a unique tiebreaker, equal-priority items have non-deterministic extraction order. In trading systems (FIFO execution), simulation (reproducibility), and testing (regression stability), this is a correctness bug — not a style preference. Note: `priority_queue` is not stable — equal-priority elements do NOT preserve insertion order. A sequence counter tiebreaker is the standard fix.

</> Storing Pairs, Tuples & Custom Structs

```
// Pairs: lexicographic (by .first, then .second)
std::priority_queue<std::pair<int, std::string>>
    pq;
pq.push({3, "low"});
pq.push({10, "high"});
pq.top(); // {10, "high"}
```

```
// Tuples: lexicographic comparison
using Entry = std::tuple<int, int64_t, int>;

std::priority_queue<Entry,
    std::vector<Entry>,
    std::greater<Entry> // min-heap
> event_queue;

// Push: (priority, timestamp, id)
event_queue.push({5, ts_now(), 42});
auto [pri, ts, id] = event_queue.top();
```

```
struct Task { // priority, enqueue_time, task_id

    bool operator<(const Task& o) const {
        if (priority != o.priority) return priority < o.priority;           // higher priority first
        if (enqueue_time != o.enqueue_time) return enqueue_time > o.enqueue_time; // FIFO on tie
        return task_id > o.task_id;                                           // deterministic
    }
};

std::priority_queue<Task> tasks; // uses operator< → max-heap on priority
```



Practical Patterns



Dijkstra's Algorithm

Min-heap on (dist, node).
Extract closest unvisited.
 $O((V+E) \log V)$.



Top-K Maintenance

Min-heap of size k .
Push; pop if size $> k$.
 $O(n \log k)$ total.



Event-Driven Simulation

Min-heap on event time.
Process earliest, generate
new events.



Task Scheduler

Priority by deadline.
 $O(\log n)$ insert and
extract-next.



Streaming Selection

Running top- k from
unbounded stream. Fixed
memory footprint.



Trading: Best-Candidate

Best bid/ask at top.
Captures the retrieval
intuition, not full book.

Trading nuance: priority_queue captures the best-candidate retrieval intuition. Real order books need richer structures — cancel/modify support, per-price-level FIFO queues, and level iteration — so production books use specialized designs. But the conceptual model starts here.

Code: Dijkstra's Algorithm

```
// dijkstra(const Graph& g, int src) -> vector<int>
using PII = std::pair<int,int>; // {dist, node}
std::vector<int> dist(g.n, INT_MAX);
// Min-heap on distance
std::priority_queue<PII,
    std::vector<PII>,
    std::greater<PII>> pq;

dist[src] = 0;
pq.push({0, src});

while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue; // stale!
    for (auto [v, w] : g.adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v}); // lazy
        }
    }
}
```

Lazy deletion pattern: push duplicates with new distances, skip stale entries ($d > \text{dist}[u]$) on extraction. Avoids needing decrease-key.

Code: Top-K Maintenance

```
// Top-K largest elements from stream
template<typename T>
class TopK {
    int k_;
    // Min-heap: smallest of top-k at root
    std::priority_queue<T,
        std::vector<T>,
        std::greater<T>> heap_;
public:
    explicit TopK(int k) : k_(k) {}

    void insert(const T& val) {
        if ((int)heap_.size() < k_) {
            heap_.push(val);
        } else if (val > heap_.top()) {
            heap_.pop();
            heap_.push(val);
        }
        // else: val too small, ignore
    }
}
```

To find k LARGEST, use a MIN-heap of size k. The smallest of the k largest sits at root. New element > top? Pop and push. $O(n \log k)$ time, $O(k)$ memory.



Case Study: Event-Driven Simulation

Push initial
events

Pop earliest
event

Process it

Generate
follow-ups

Repeat until
empty

```
struct Event {
    double time;
    int priority;           // type-based ordering
    uint64_t seq;          // monotonic tiebreaker
    std::function<void()> action;

    bool operator>(const Event& o) const {
        if (time != o.time) return time > o.time;
        if (priority != o.priority)
            return priority > o.priority;
        return seq > o.seq;    // deterministic
    }
};

std::priority_queue<Event, std::vector<Event>,
    std::greater<Event>> sim_queue;
```

What Makes This Work

- Min-heap on event time
- Deterministic tie-breaking by priority then sequence
- Lazy cancellation for removed events
- $O(\log n)$ per event insert/extract
- Replay produces identical results
- Used in backtesting, network sim, game engines

This pattern connects all the ideas: min-heap, custom comparator, tie-breaking, and lazy deletion.



What `priority_queue` Cannot Do



No Arbitrary Deletion

Can't remove a specific element. Only `pop()` removes the top.



No Decrease-Key

Can't change priority of an existing element in-place.



No Ordered Iteration

Underlying container is heap-ordered, not sorted.



No Find / Contains

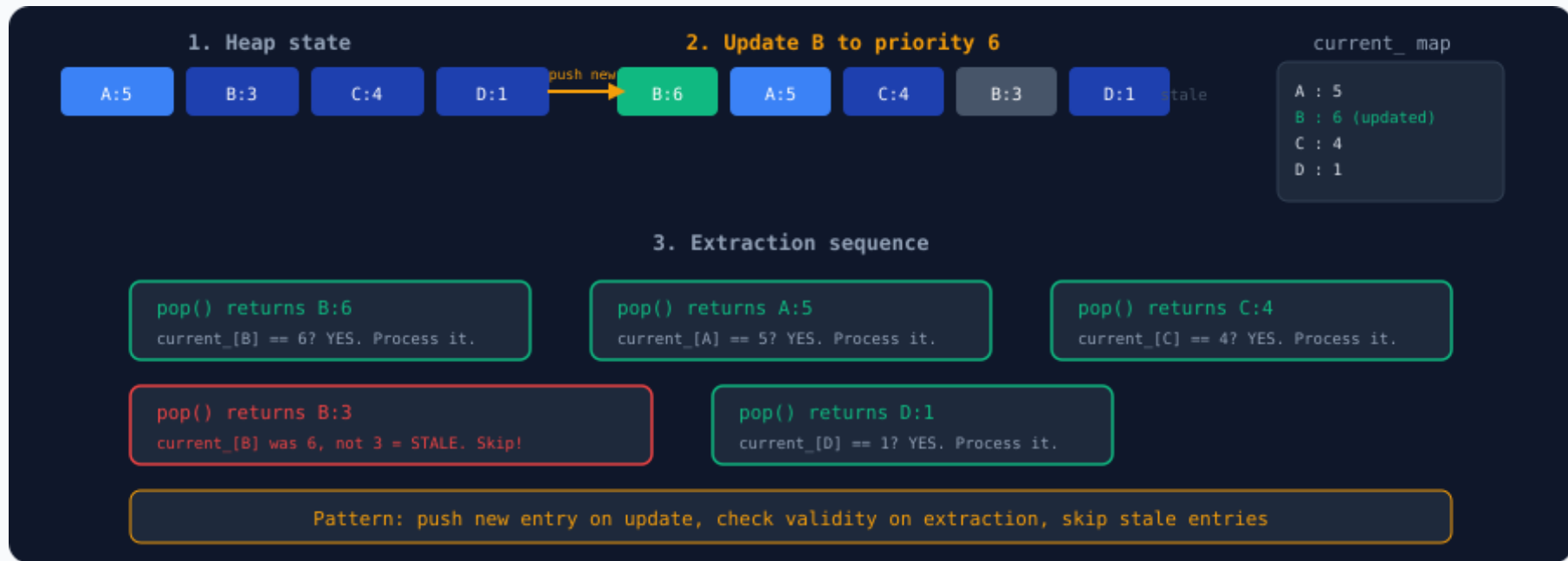
No way to check membership without $O(n)$ scan.

These are design tradeoffs, not bugs.

The restricted API makes the common case fast. If you need the missing operations frequently, the right answer is a different data structure — not hacks on top of `priority_queue`. Also: `priority_queue` is not thread-safe — concurrent push/pop requires external locking (`std::mutex`) or a lock-free alternative.



How Engineers Handle It: Lazy Deletion



Other Workarounds

External index map (track current priority per ID) **Periodic rebuild** (drain, filter stale, make_heap) **Switch to set** (when updates >> extractions)



Alternatives Comparison

Structure	Insert	Extract Best	Find/Delete	Iterate Sorted	Memory
priority_queue	$O(\log n)$	$O(\log n)$	$O(n)$	No	Contiguous
set / multiset	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes	Scattered
sorted vector	$O(n)$	$O(1)^*$	$O(\log n)$	Yes	Contiguous
vector + heap algs	$O(\log n)$	$O(\log n)$	$O(n)$	No	Contiguous
vector + re-sort	$O(1)+O(n \lg n)$	$O(1)^*$	$O(\log n)$	Yes	Contiguous

Why Not Just Sort a Vector?

- If you insert often + extract best often → re-sorting is $O(n \log n)$ per batch vs $O(\log n)$ per operation
- If data is mostly static + you need many lookups → sorted vector with binary search is often better
- Heap wins when the operation mix is "many inserts, many extracts, few lookups"

Quick Decision Guide

- **priority_queue** Insert/extract-best dominates
- **set / multiset** Need find, erase, or iteration
- **vector + heap algs** make/push/pop/sort/is_heap
- **sorted vector** Mostly static, many lookups

* $O(1)$ extract-best from sorted vector = `pop_back()`. Maintaining sort costs $O(n)$ per insert. `vector + heap algs` = `std::make_heap`, `push_heap`, `pop_heap`, `sort_heap`, `is_heap` — same structure as `priority_queue` but with direct container access. C++20 adds `constexpr` and `std::ranges` heap algorithms.



Common Mistakes & Pitfalls



```
// Expecting sorted iteration:  
for (auto x : pq_internals)  
    std::cout << x; // NOT sorted!
```



```
// Must pop() one at a time:  
while (!pq.empty()) {  
    process(pq.top()); pq.pop();  
}
```



```
// Using <= in comparator:  
auto cmp = [](int a, int b) {  
    return a <= b; // UB!  
};
```



```
// Use strict < or >:  
auto cmp = [](int a, int b) {  
    return a < b; // correct  
};
```



```
// Trying to erase an element:  
pq.erase(some_value); // NO  
// Does not compile
```



```
// Use lazy deletion instead:  
deleted.insert(some_value);  
// Skip in pop loop
```

A heap is NOT a sorted structure. Only top() is guaranteed. Everything else is "heap order" — weaker than sorted.



What Engineers Care About



Operation Mix

What's the workload? Mostly insert/extract, or lots of find/delete/update? The right container depends on the operation profile, not just one operation's complexity.



Asymptotics vs Constant Factors

Same $O(\log n)$ doesn't mean same speed. Cache locality, pointer overhead, and branch behavior create 2-5x differences in practice. Know when constants matter.



Correctness Under Ties

Equal-priority items need deterministic ordering for reproducibility. Always add a unique tiebreaker. This isn't optional in trading or simulation.



Mutable Priority Limitations

Can you live without decrease-key? If yes, `priority_queue` + lazy deletion. If updates dominate, consider set or an indexed priority queue.



Choosing the Right Container

Don't reach for the fanciest structure. A `priority_queue` that fits the workload beats a red-black tree that doesn't. Explain your reasoning.

Not interview questions — but the reasoning lens that separates a strong answer from syntax recitation.

Summary



Use priority_queue When

- Insert + extract-best dominates
- Cache locality matters
- You don't need find / delete
- Simple, correct, fast



Consider Alternatives When

- Need arbitrary deletion
- Need decrease-key often
- Need sorted iteration
- Need membership queries



Always Remember

- Comparator = lower priority
- Strict weak ordering required
- Add deterministic tiebreakers
- Heap $\neq$ sorted structure

"The art is not in knowing the API — it's in knowing when the abstraction fits your problem."